

Imbalance-XGBoost: leveraging weighted and focal losses for binary label-imbalanced classification with XGBoost^{☆,☆☆}

Chen Wang^a, Chengyuan Deng^a, Suzhen Wang^{b,*}

^a Department of Computer Science, Rutgers University, Piscataway, NJ 08854, USA

^b Department of Health Statistics, Weifang Medical University, Weifang, Shandong 261053, China

ARTICLE INFO

Article history:

Received 9 November 2019

Revised 27 March 2020

Accepted 31 May 2020

Available online 2 June 2020

Keywords:

Imbalanced classification

XGBoost

Python package

ABSTRACT

The paper presents *Imbalance-XGBoost*, a Python package that combines the powerful XGBoost software with weighted and focal losses to tackle binary label-imbalanced classification tasks. Though a small-scale program in terms of size, the package is, to the best of our knowledge, the first of its kind which provides an integrated implementation for the two loss functions on XGBoost and brings a general-purpose extension to XGBoost for label-imbalanced scenarios. In this paper, the design and usage of the package are discussed and illustrated with examples. Furthermore, as the first- and second-order derivatives of the loss functions are essential for the implementations, the algebraic derivation is discussed and it can be deemed as a separate contribution. The performances of the methods implemented in the package are extensively evaluated on Parkinson's disease classification dataset, and multiple competitive performances are presented with the ROC and Precision-Recall (PR) curves. To further assert the superiority of the methods, the performances on four other benchmark datasets from the UCI machine learning repository are additionally reported. Given the scalable nature of XGBoost, the package has great potentials to be broadly applied to real-life binary classification tasks, which are usually of large-scale and label-imbalanced.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

XGBoost is an advanced Gradient Tree Boosting-based software that can efficiently handle large-scale Machine Learning tasks [4]. Merited by its performance superiority and affordable time and memory complexities, it has been widely applied to a variety of research fields since been proposed, ranging from cancer diagnosis [36] and medical record analysis [35] to credit risk assessment [3] and metagenomics [38]. Also, because of its easy-to-use Python interface and explainable nature, it has become the de facto method-of-the-first-choice for a majority of data science problems. For instance, in the famous *Kaggle* (<https://www.kaggle.com/competitions>) competitions, many winning teams built their models based on XGBoost and expressed positive views on the method and its variations [4,24,25]. It could be tentatively pre-

dicted that in the near future, XGBoost and its variations will remain one of the most-applied methods in the data science community.

On the other hand, although XGBoost has achieved considerable success on both regression and classification problems, its performance often becomes subtle when a situation of label-imbalanced classification is brought up. The problem of classification under label-imbalanced situations exists commonly in practice, and most accuracy-driven 'vanilla' Machine Learning methods suffer performance decline if the ratio between labels becomes biased. For example, in cancer diagnosis, it is common to have 95% of the patients without cancer and only 5% with it. If the model simply predict everyone as 'no cancer', then the accuracy is 95%, which is remarkably high. However, missing to spot any cancer patient can lead to fatal consequences.

There have been mixed reports on the capabilities of XGBoost in handling label-imbalanced data. For example, Zhao et al. [42] demonstrated through their experiments that XGBoost can outperform other methods on skewed datasets, while the figures in [20] suggested that vanilla XGBoost must be combined with other ensembling methods to achieve satisfactory results. Given the popularity of XGBoost and the fact that label-skewed data is, unfortunately, commonly encountered in practice, this performance decay

[☆] This work is supported in part by National Natural Science Foundation of China (Grant 81872719, 81803337); Provincial Natural Science Foundation of Shandong Province (Grant ZR201807090257) and National Bureau of Statistics Foundation Project (Grant 2018LY79).

^{☆☆} Handling Editor: Jiwen Lu.

* Corresponding author.

E-mail address: wangsz@wfmec.edu.cn (S. Wang).

will leave significant negative effects on related research and applications.

This paper introduces *imbalance-XGBoost*, an XGBoost-based Python package addressing the label-imbalanced issue in the binary label regime by implementing weighted (cross-entropy) and focal losses on the boosting machine. Weighted cross-entropy loss is one of the simplest algorithm-level cost-sensitive methods [32] for learning imbalanced data. It follows the straightforward idea to increase the penalization of the misclassification of a certain class, and it has been widely applied to the label-imbalanced domain [13]. In contrast, focal loss [17] is a relatively novel method originated from research in object detection. The idea of the method is to add a $(1 - y_j^\gamma)$ factor to the cross-entropy function (where y_j is the prediction of $p(x = j)$), and this will reduce the importance of the well-classified data points. Comparing with weighted cross-entropy, focal loss enjoys a more robust parameter configuration as the method will work in our favor as long as $\gamma > 0$.

To the best of our knowledge, there has not been significant publication discussing the implementation of the two losses on XGBoost previously. Existing studies on XGBoost under label-imbalanced scenarios usually adopt data-level approaches such as re-sampling [14] and/or cost-sensitive loss with non-trainable *a priori* modifications [39]. Chen et al. [5] mentioned weighted XGBoost in their work, but details regarding the implementation are not presented. A major challenge in applying the two loss functions to XGBoost is that to approximate the incremental learning objective, first- and second-order derivatives of the loss functions with respect to the predictions must be presented (See Section 3 for details). And an algebraic contribution of this paper is the derivations and implementations of the derivatives that enable the two losses to be run with XGBoost.

The package is written in Python with hard dependency on XGBoost, Numpy [34], and Scikit-learn [26]. The losses are integrated into the XGBoost system by the customized loss framework of the software, provided the derived expressions of the derivatives. Since the major methods in the program are included in the dependency graph, the core part of the package is of small scale, with only a few hundred lines of codes. Nevertheless, the function derivatives and implementations and the significance in practical applications make the work non-trivial. The software has been released on Github¹ and PyPI,² and it has started to attract users with considerable downloads.

The rest of the paper goes as follows. Section 2 introduces the package from the perspectives of toolkit designers and users; Section 3 provides the theoretical foundation of the second-order approximation of gradient boosting trees used in XGBoost and the first- and second-order derivatives of the two losses; Some related studies are surveyed and discussed in Section 4, then the experimental performances of the package are empirically examined on five imbalanced binary classification datasets in Section 5; And finally, Section 6 gives a general conclusion of the paper.

2. Design and usage of imbalance-XGBoost

2.1. Code design

Though the XGBoost method has implementations in multiple languages, Python is picked as the language-of-choice for its wide recognition and application in data science. The codes follow the standard of PEP8, and the project has been designed as open-source with codes on the Github page. We strive to keep the

program consistent with 'standard' practices in Python-based data science. The input data is designated as Numpy array [34], but by explicitly adding `np.array()` conversion, data types compatible with Numpy array (e.g. Pandas Dataframe [22]) will also work on the package.

The overall program is consist of three classes: one main class `imbalance_xgboost`, which contains the method the users will be applying, and two customized-loss classes, `Weight_Binary_Cross_Entropy` and `Focal_Binary_Loss`, on which the imbalanced losses are based. The loss functions are designed as separate classes for the convenience of parameter tuning, and they are not supposed to be called by the users. When initializing an `imbalance_xgboost` object, keyword `special_objective` will be recorded by the `__init__()` method. Then, when executing the `fit()` function, the corresponding loss function object will be instantiated with the given parameter, and the built-in `xgb.train()` method of XGBoost will be able to fit the model based on the customized loss function. Fig. 1 illustrates the overall structure of the program.

The package is designed to be an estimator class of the *Scikit-learn* toolkit. This scheme enables the model and parameter selection methods in *Scikit-learn*, such as `GridsearchCV()` and `RandomizedSearchCV()`, to be directly applied to find the best parameters for imbalanced XGBoosts. In practical data science tasks, this feature is crucial as the optimal models rely heavily on parameter tuning and selection. Also, estimator in *Scikit-learn* can be combined with other estimators (transformers) by integrating them to a Pipeline object [26]. This allows the weighed- and focal-XGBoost to be easily combined with other pre-processing methods, such as resampling, to produce more robust results. Section 2.2 will provide more details for the package to tune parameters and perform cross-validation with *Scikit-learn*.

2.2. Model optimization and evaluation with Scikit-learn

Similar to common classifiers in *Scikit-learn*, the best classifier/parameter can be obtained through exhaustive or random search with the functions `GridsearchCV()`. It is noticeable that after fitting the model, it is possible to retrieve the 'plain' booster by accessing member `opt_booster.booster`, and the object will be a XGBoost class (instead of Imbalance-XGBoost class). This makes it possible for the user to train the model on a machine where *Imbalance-XGBoost* is available, save the model as 'plain' XGBoost, and run on another machine where only the original XGBoost package is installed.

Likewise, the cross-validation evaluation of *Imbalance-XGBoost* is similar to the process on an 'ordinary' classifier, and one can simply follow the documentation of *Scikit-Learn* to perform it. Notice that if one intends to perform a combination of parameter selection and cross-validation, it is recommended that a new booster with random initialization and selected parameters should be instantiated after the parameter selection, instead of directly using the fitted optimal booster from `GridsearchCV()`. The reason for doing so is that for the iterative training process of XGBoost, a booster trained from a randomized state is essential for a fair evaluation.

2.3. Built-in evaluation score

There are three evaluation functions in the package. The overriding `score()` function serves the purpose to evaluate prediction accuracy under the format of predictions, which are pre-sigmoid values (in range $[-\inf, +\inf]$) by default, by wrapping the sigmoid transformation and accuracy checking together. In comparison, function `score_eval_func()` is the method to return metrics other than accuracy. In label-imbalanced binary classification,

¹ <https://github.com/jhwjh0123/Imbalance-XGBoost>.

² <https://pypi.org/project/imbalance-xgboost/>.

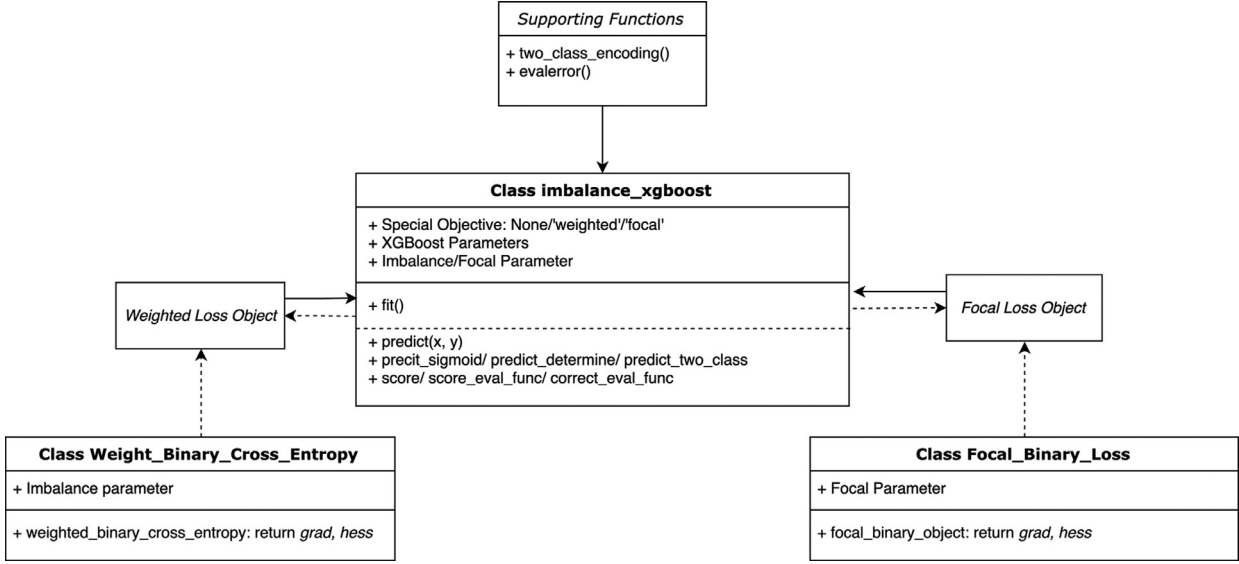


Fig. 1. The overall structure of the program.

accuracy cannot reliably reveal the performance quality on its own as the metric can be 'tricked' by predicting all the instances as the majority class. This type of prediction will lead to high accuracy, yet the classifier actually does nothing. Thus, metrics taking 'preciseness' into accounts, such as precision, recall, F_1 score and Matthew's Correlation Coefficient (MCC) [21], are often applied for the scenario [29]. Function `score_eval_func()` provides implementations for all the metrics mentioned above, and it can be overloaded by specifying the partial argument 'mode'(which can be accomplish by `functools.partial()`).

3. Theories and derivatives

In this section, the mathematical foundations and derivations for the loss functions are discussed. For a high-level introduction, since XGBoost adopts an additive learning scheme with a second-order approximation, the first-order derivative (short-handed as 'gradient') and second-order derivative (noted as 'hessian' although somehow a misnomer) of the loss functions with respect to the prediction are required for fitting the model. To illustrate a clear mechanism, the section will first review the second-order approximation of additive tree boosting in Section 3.1. Subsequently, the derivatives of gradients and Hessians of the weighted and focal losses will be discussed in Section 3.2 and 3.3, respectively.

The notations used in this section will be as follows. We use m to denote the number of data and n for the number of features. The 'raw prediction' before the sigmoid function will be denoted as z_i , and the probabilistic prediction will be $\hat{y}_i = \sigma(z_i)$, where $\sigma(\cdot)$ represents the Sigmoid function. It is important to keep in mind that there is a discrepancy between the notations of this paper and the original XGBoost literature [4], as the $\hat{y}$ in their analysis is denoted as z here. y_i is used to denote the true label, and α and γ are used for the parameters for the two loss functions, respectively.

3.1. Second-order approximation of gradient boosting tree

Denoting the input as $\mathbf{x}_i$ and the target as y_i , according to [4], the additive learning objective used in practice is:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, z_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (1)$$

Eq. (1) is a *regularized objective*, where $l(\cdot, \cdot)$ denotes the loss between the targets (y_i) and predictions (z_i), and $\Omega(\cdot)$ de-

notes the complexity of the model. The tree is fitted in an additive manner with $z_i^{(t)} = z_i^{(t-1)} + f_t(\mathbf{x}_i)$, and t stands for the t -th iteration of the training process. Notice that the notations used above are slightly different from the original literature. Our goal is to find the $f_t(\cdot)$ of the t -th iteration to optimize objective 1; To achieve this, applying a second-order Taylor expansion on Eq. (1), one will get:

$$\begin{aligned} \mathcal{L}^{(t)} &\approx \sum_{i=1}^n \left[l(y_i, z_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i (f_t(\mathbf{x}_i))^2 \right] + \Omega(f_t) \\ &\propto \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i [f_t(\mathbf{x}_i)]^2 \right] + \Omega(f_t) \end{aligned} \quad (2)$$

The last line comes from the fact that the $l(y_i, z_i^{(t-1)})$ term can be removed from the learning objective as it is unrelated to the fitting of the model in the t -th iteration. In Eq. (2), there are $g_i = \frac{\partial L}{\partial z_i}$ and $h_i = \frac{\partial^2 L}{\partial z_i^2}$, which are the 'gradient' and 'hessian' terms mentioned before. Notice that both g_i and h_i are scalars, as individual boosting trees only deal with binary problems. Multi-class classification tasks are usually processed by an ensemble of binary classification trees (so-called *one-vs-all* scheme, see [2,8]).

Since XGBoost does not provide automatic differentiation, the hand-derived derivatives will be essential. Meanwhile, the derived expressions have further potentials to be applied into other machine learning tasks. Therefore, the derivatives are discussed in Sections 3.2 and 3.3. For both loss functions, sigmoid is selected as activation, and the following basic property of sigmoid will be consistently used in the derivatives:

$$\begin{aligned} \frac{\partial \hat{y}}{\partial z} &= \frac{\partial \sigma(z)}{\partial z} \\ &= \sigma(z)(1 - \sigma(z)) \\ &= \hat{y}(1 - \hat{y}) \end{aligned} \quad (3)$$

3.2. Weighted cross-entropy loss

The weighted cross-entropy loss for binary classification can be denoted as follows:

$$L_w = - \sum_{i=1}^m (\alpha y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \quad (4)$$

where α indicates the ‘imbalance parameter’. Intuitively, if α is greater than 1, extra loss will be counted on ‘classifying 1 as 0’; On the other hand, if α is less than 1, the loss function will weight relatively more on whether data points with label 0 are correctly identified.

The first order derivative is presented as follows:

$$\frac{\partial L_w}{\partial z_i} = -\alpha^{y_i}(y_i - \hat{y}_i) \quad (5)$$

The derivative is similar with the $\partial L/\partial z$ term for ordinary cross-entropy loss. A notable difference is that a α^{y_i} term is added to control the present of the parameter.

Taking derivative with respect to z_i again, one will get the second-order derivative:

$$\frac{\partial^2 L_w}{\partial^2 z_i} = \alpha^{y_i}(1 - \hat{y}_i)(\hat{y}_i) \quad (6)$$

After plugging Eq. (3) to the derivation.

3.3. Focal loss

According to Lin et al. [17], the binary focal loss can be denoted as:

$$L_f = -\sum_{i=1}^m y_i(1 - \hat{y}_i)^\gamma \log(\hat{y}_i) + (1 - y_i)\hat{y}_i^\gamma \log(1 - \hat{y}_i) \quad (7)$$

As one can observe, if one sets $\gamma = 0$, the equation will become ordinary cross-entropy loss. Taking Eq. (3) into consideration, the first derivative of the focal loss can be denoted as:

$$\frac{\partial L_f}{\partial z_i} = \gamma[(\hat{y}_i + y_i - 1)(y_i + (-1)^{y_i}\hat{y}_i)^\gamma \log(1 - y_i - (-1)^{y_i}\hat{y}_i) + (-1)^{y_i}(y_i + (-1)^{y_i}\hat{y}_i)^{\gamma+1}] \quad (8)$$

And if γ is set to 0 in Eq. (8), the derivative will be the same as cross-entropy loss. The equation follows a clear structure, but it is still lengthy. To simplify the expression, one can set the following short-hand variables:

$$\begin{cases} \eta_1 = \hat{y}_i(1 - \hat{y}_i) \\ \eta_2 = y_i + (-1)^{y_i}\hat{y}_i \\ \eta_3 = \hat{y}_i + y_i - 1 \\ \eta_4 = 1 - y_i - (-1)^{y_i}\hat{y}_i \end{cases} \quad (9)$$

Plugging these representations into Eq. (8), the first-order derivative can be denoted as:

$$\frac{\partial L_f}{\partial z_i} = \gamma \eta_3 \eta_2^\gamma \log(\eta_4) + (-1)^{y_i} \eta_2^{\gamma+1} \quad (10)$$

Finally, taking derivatives with respect to z_i , and combining with Eq. (3) and (10), one can get the second-order derivative (‘hessian’), which can be denoted as:

$$\frac{\partial^2 L_f}{\partial z_i^2} = \eta_1 \left\{ \gamma \left[(\eta_2^\gamma + \gamma(-1)^{y_i} \eta_3 \eta_2^{\gamma-1}) \log(\eta_4) - \frac{(-1)^{y_i} \eta_3 \eta_2^\gamma}{\eta_4} \right] + (\gamma + 1) \eta_2^\gamma \right\} \quad (11)$$

Again, if $\gamma = 0$, the second-order derivative becomes $\eta_1 = \hat{y}_i(1 - \hat{y}_i)$, which matches the formula of the second-order derivative of ordinary cross-entropy.

4. Related work

The paper is built on the foundation of the original papers of XGBoost [4] and focal loss [17], and the methodology to program customized loss function is provided in the software’s Github

page.³ XGBoost is based on the algorithm of gradient tree boosting [9], and this method has been deemed as a powerful Machine Learning technique long before the XGBoost was born [23]. Besides XGBoost, there are other implementations of gradient boosting, such as pGBRT [33], LightGBM [15], and CatBoost [30]. Some of the implementations have additional features and are able to outperform XGBoost on some specific problems, but XGBoost remains the method-of-the-first-choice in the data science community at large. As for the recently proposed focal loss, studies related to it are usually affiliated with Neural Networks and Deep Learning [6,37,41]. The loss function is usually applied in an end-to-end manner with automatic differentiation, and to the best of our knowledge, there has not been any notable publication comprehensively discussing the derivatives of the loss function (despite the first-order derivative was briefly discussed in another form in the original paper).

Previous applications of XGBoost in label-imbalanced scenarios focus mostly on data-level algorithms. For example, Kabir and Ludwig [14] applies several commonly-used data resampling methods before using XGBoost for label-imbalanced breast cancer classification, and He et al. [12] utilized a more advanced under-sampling method called BalanceCascade [18] with XGBoost for credit scoring. Among the limited number of publications discussing algorithm-level modification for XGBoost in imbalanced classification, Xia et al. [39] used a *a priori* modification of the sigmoid activation to achieve a better result, but the loss function was unchanged. As it has been mentioned in Section 1, [5] is by far the only implementation explicitly applied weighted function to XGBoost to best of our knowledge. It is noticeable that a Tensorflow-based gradient boosting implementation called *Tf Boosted Trees* [28] is able to run with the loss functions without the derivatives provided in this paper as it has an automatic differentiation mechanism. Nevertheless, it is a less popular package without supports of large-scale Machine Learning and compatibility with *Scikit-learn* toolkit.

As a common issue frequently encountered in practice, label-imbalanced classification has been intensively studied by researchers and there are multiple existing software programs designed to handle the problem. For a great example, [16] provides an integrated Python package called *Imbalanced-learn* for data-level resampling for imbalanced classification, and it has similar counterparts in the regime of other programming languages, such as *ROSE* in R [19]. It is worth noting that the *Imbalanced-learn* package can be considered as an extension of *Scikit-learn*, and the Machine Learning toolkit itself also provides elementary methods to deal with label-imbalanced problems [26]. Other software programs concerning label-imbalanced classification include popular Data Mining toolkits, such as KEEL [1] and WEKA [11]. In addition, Zhang et al. [40] provides a software containing a set of algorithms specifically for multi-class label-imbalanced problems, serving as one of the most recent studies on this topic.

5. Experiments

In this section, experimental results of the proposed Imbalance-XGBoost package are demonstrated in two parts: (1) Binary classification on Parkinson Disease data, a recently-collected imbalance dataset; (2) Binary classification on four benchmark datasets (with imbalance ratio) published in UCI Machine Learning Repository [7]: *ecoli*(9:1), *arrhythmia*(17:1), *car_eval_4*(26:1) and *ozone*(42:1). To provide an observation of the performance improvement particularly by two loss functions, we compare the results of Weighted-XGBoost and Focal-XGBoost with vanilla XGBoost using the same

³ <https://github.com/dmlc/xgboost/blob/master/demo/guide-python/custom-objective.py>.

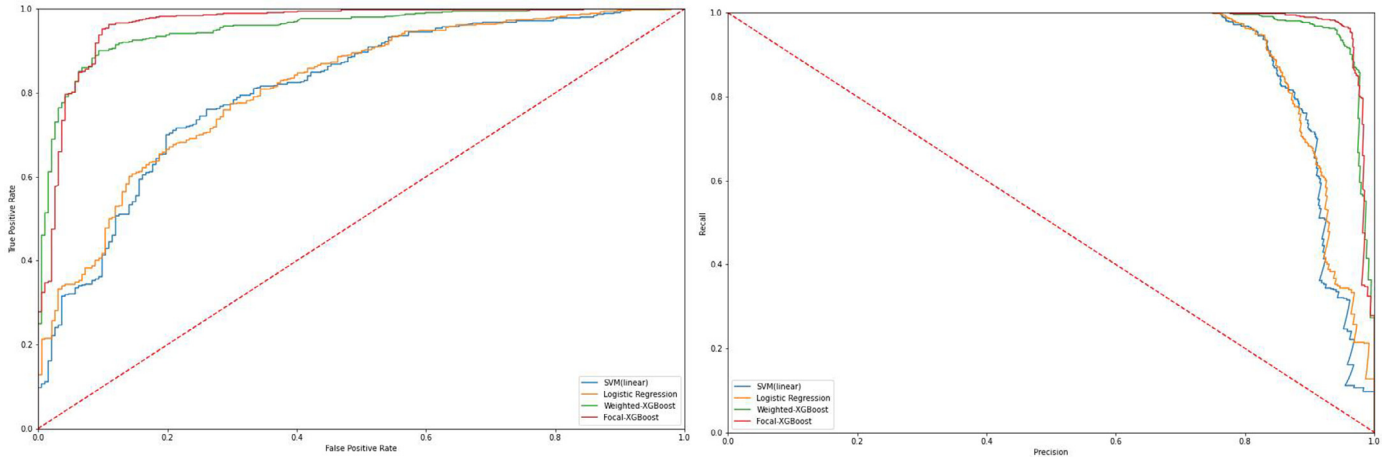


Fig. 2. ROC and Precision-Recall Curve of Baseline Feature.

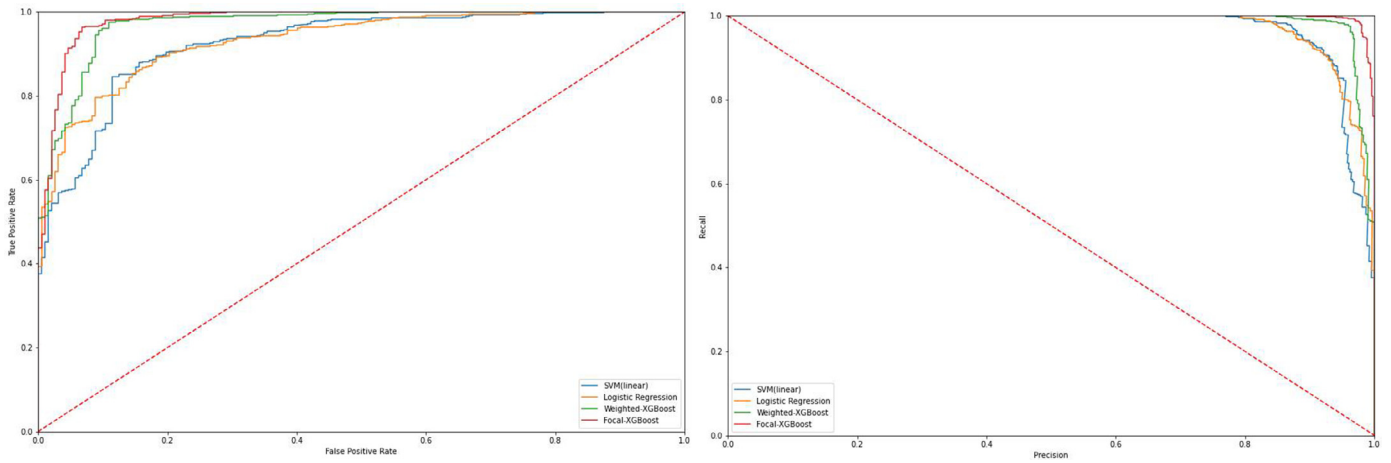


Fig. 3. ROC and Precision-Recall Curve of MFCC Feature.

parameter set, and the performances are also compared against SVM and Logistic regression.

5.1. Parkinson disease dataset and setup

The Parkinson's Disease(PD) classification data⁴ is a recently introduced dataset [31] with 757 features categorized into 7 specific groups (two originally separate groups, Bandwidth and Formant, are merged in experiments), the data was gathered from 188 Parkinson's disease patients and 64 healthy individuals at the Department of Neurology in Cerrahpasa Faculty of Medicine, Istanbul University. Each individual corresponds to 3 records, and due to the differences between the number of participants of the two sides, a label imbalance ratio of 188:64 (roughly 3:1) emerges.

The original literature [31] demonstrated the classification performances with seven conventional classification algorithms and two ensemble approaches. The experiments in the original literature were conducted with leave-one-object-out cross validation. To keep consistent with the original system, the same cross validation setup is applied in this paper. Furthermore, the results in [31] illustrate a high accuracy and relatively lower F_1 score, indicating that the classifiers failed to tell the two classes clearly and likely

achieved the performance by overwhelming predicting the majority class. This is an unfavourable behavior in label-imbalanced classification, and one advantage of *Imbalance-XGBoost* is that it does not suffer from this problem. This characteristic can be further verified by the ROC and Precesion-Recall (PR) curves in Figs. 2–4.

To conduct the leave-one-object-out cross validation, the correctness collection function mentioned in Section 2.3 is applied. By collecting results of True_Positive (TP), True_Negative (TN), False_Positive (FP), and False_Negative (FN), the confusion metric can be obtained and accuracy and F_1 score are computed accordingly. The records are evaluated in a *per-record* manner, which means the 3 records of one object (patient/healthy individual) will be evaluated individually, and 3 counts of the correctness will be added.

5.2. Classification results and discussion

Accuracy and F_1 score of the test set with 6 sets of features are presented in Table 1, where Best in [31] indicates the best performance of accuracy and F_1 score retrieved from the paper.

Without exception, although a slight declination of accuracy could be observed in weighted-XGBoost and focal-XGBoost, both classifiers generate a significantly higher F_1 score. The increase of F_1 score and the decrements of accuracy suggest that the previously-obtained higher accuracy is a consequence of overlooking minority class. Furthermore, for almost all the feature groups, the high-

⁴ available publicly, url: <https://archive.ics.uci.edu/ml/datasets/Parkinson%27s+Disease+Classification>.

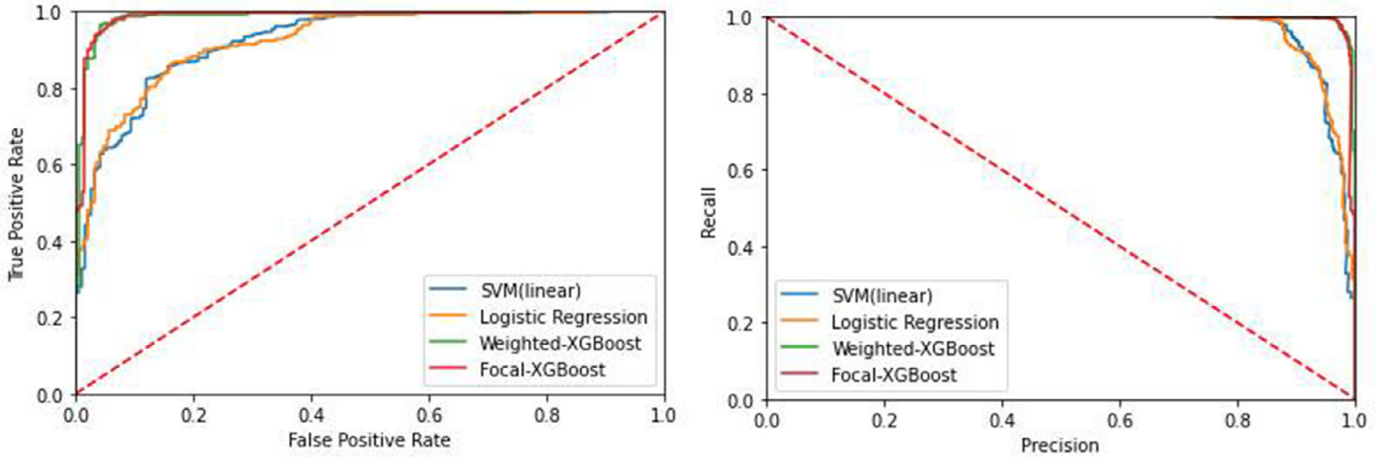


Fig. 4. ROC and Precision-Recall Curve of Top50 Features.

Table 1

Classification performance on individual features.

	Baseline features		MFCC	
	Accuracy	F_1 score	Accuracy	F_1 score
Best in [31]	0.79	0.75	0.84	0.83
XGBoost	0.76	0.84	0.80	0.87
Weighted-XGBoost	0.76	0.85	0.80	0.87
Focal-XGBoost	0.76	0.85	0.82	0.89
	Wavelet Features		Bandwidth + Formant	
	Accuracy	F_1 score	Accuracy	F_1 score
Best in [31]	0.78	0.74	0.77	0.72
XGBoost	0.72	0.82	0.71	0.83
Weighted-XGBoost	0.75	0.85	0.74	0.85
Focal-XGBoost	0.75	0.85	0.75	0.85
	Intensity Based		Vocal Fold-Based	
	Accuracy	F_1 score	Accuracy	F_1 score
Best in [31]	0.77	0.74	0.77	0.74
XGBoost	0.73	0.83	0.72	0.82
Weighted-XGBoost	0.75	0.85	0.75	0.84
Focal-XGBoost	0.75	0.85	0.76	0.85

Table 2

Classification performance on top 50 features.

	Top 50 Features	
	Accuracy	F_1 score
Best in [31]	0.86	0.84
XGBoost	0.83	0.89
Weighted-XGBoost	0.82	0.88
Focal-XGBoost	0.84	0.90

est F_1 score is obtained by focal-XGBoost. This observation can be explained from an algorithmic perspective that focal loss is more robust to parameters, while weighted loss is prone to the effect of sub-optimal parameters even if parameter search is applied.

Sakar et al. [31] also applied a classifier with 50 top-ranked features selected by mRMR (minimum Redundancy-Maximum Relevance). The feature selection technique is based on the principle of maximizing the joint dependency of top ranking variables on the targeted one by reducing the redundancy among them [10,27]. For a comparison purpose, this paper employs the same technique with provided Python interface,⁵ and produces a subset of top-50 features to run with *Imbalance-XGBoost*. The performance of weighted- and focal-XGBoost on the top-50 features can be observed in Table 2.

Consistent with the performance on individual groups of features, focal-XGBoost classifier has the highest F_1 score, slightly better than weighted-XGBoost. Both weighted- and focal-XGBoost outperform best classifier in [31] by a large margin, and since the top-50 feature can be regarded as a ‘master subset’, the superiority of the methods implemented in imbalance-XGBoost can be further corroborated.

To provide further insights for the predictions of weight- and focal-XGBoosts, the ROC and Precision-Recall (PR) curves of the two boosting trees and two other methods (SVM and Logistic Regression, which are used in [31]) are illustrated in Figs. 2–4. Limited by the space, only the feature sets of Baseline, MFCC and Top 50 are adopted. From the figures, it can be observed that the XGBoost-based models consistently provide superior performance in terms of AUC and PR curve. This further validate our earlier assertion that the higher accuracy of the method in [31] comes from mis-classifying the minority instances into the majority class.

5.3. Classification for benchmark imbalanced datasets

To further evaluate the performance of Imbalance-XGBoost, we introduce four other imbalanced datasets for binary classification from UCI Machine Learning Repository: *ecoli*, *arrhythmia*, *car_eval_4* and *ozone*. Notice that the Parkinson Disease dataset has an imbalance ratio of 3:1, while these four datasets have higher ratios progressively as 9:1 (*ecoli*), 17:1 (*arrhythmia*), 26:1 (*car_eval_4*), 42:1 (*ozone*). Slightly different from the experiments on Parkinson Dataset, we apply K-Fold(K=5) cross validation for Grid Search instead of leave-one-out, on the grounds that these four datasets include a considerable number of instances and K-Fold is commonly used in practice.

Our focus here is to illustrate the effectiveness of the weighted and focal loss functions, so we compare weighted- and focal-XGBoosts against its vanilla counterpart. The classification results are presented in Table 3, with accuracy, F_1 score and MCC(Matthews Correlation Coefficient) as evaluation metrics.

From the table, it can be found that the classification accuracy of weighted- and focal-XGBoost is at least as competitive as XGBoost. In terms of F_1 score and MCC, which are considered the most crucial metrics for imbalanced classification, weighted- and focal-XGBoost outperforms vanilla XGBoost by a large margin on all datasets. We also remark that as the imbalance ratio goes up, the improvements on F_1 score and MCC for weighted- and focal-XGBoosts become more significant.

⁵ <https://github.com/fbrundu/pymrmr>.

Table 3
Classification performance on benchmark imbalanced datasets.

	ecoli (9 : 1)			arrhythmia (17 : 1)		
	Accuracy	F ₁ score	MCC	Accuracy	F ₁ score	MCC
XGBoost	0.926	0.627	0.586	0.958	0.627	0.605
Weighted-XGBoost	0.935	0.645	0.616	0.967	0.681	0.665
Focal-XGBoost	0.935	0.645	0.616	0.960	0.679	0.662

	car_eval_4 (26 : 1)			ozone (42 : 1)		
	Accuracy	F ₁ score	MCC	Accuracy	F ₁ score	MCC
XGBoost	0.904	0.239	0.217	0.967	0.144	0.154
Weighted-XGBoost	0.959	0.268	0.266	0.959	0.268	0.266
Focal-XGBoost	0.911	0.287	0.273	0.964	0.180	0.173

6. Conclusion

This paper presents a novel Python-based package, namely *Imbalance-XGBoost*, for binary label-imbalanced classification with XGBoost. The package implemented weighted cross-entropy and focal loss functions on XGBoost, and it is fully compatible with the popular *Scikit-learn* package in Python. The design and usage of the package are introduced, and the discussion of methods provide a clear and comprehensive user guidance. The theories and derivatives essential to the package are further discussed, and experiments based on five imbalance classification datasets are conducted with competitive performances illustrated. Overall, the package demonstrated in this paper successfully combines XGBoost with popular label-imbalance-robust loss functions and provides one of the most competitive performances up to date.

In summary, this paper has made three main contributions. Firstly, the paper has introduced a novel package that leverages the power of weighted and focal loss function for XGBoost, and it has huge potentials to be applied to a variety of real-life binary classification problems. Secondly, the paper has studied the theoretical foundations of the second-order approximation of XGBoost and has provided essential derivatives for the loss functions to be applied. The derivatives can also be applied to other fields in Machine Learning, and the equations in the merged form are convenient to be vectorized. And finally, the paper has offered new advanced performances on the five benchmark datasets, and the emphasis of the imbalanced nature provides new a perspective to study them.

Declaration of Competing Interest

The authors declare no conflict of interest.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.patrec.2020.05.035](https://doi.org/10.1016/j.patrec.2020.05.035).

References

- [1] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, L. Sánchez, F. Herrera, Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework, *J. Mult.-Valued Logic Soft Comput.* 17 (2011) 255–287.
- [2] E.L. Allwein, R.E. Schapire, Y. Singer, Reducing multiclass to binary: a unifying approach for margin classifiers, *J. Mach. Learn. Res.* 1 (Dec) (2000) 113–141.
- [3] Y.-C. Chang, K.-H. Chang, G.-J. Wu, Application of extreme gradient boosting trees in the construction of credit risk assessment models for financial institutions, *Appl. Soft Comput.* 73 (2018) 914–920.
- [4] T. Chen, C. Guestrin, Xgboost: a scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794.
- [5] W. Chen, K. Fu, J. Zuo, X. Zheng, T. Huang, W. Ren, Radar emitter classification for large data set based on weighted-xgboost, *IET Radar Sonar Navig.* 11 (8) (2017) 1203–1207.
- [6] I. Demir, K. Koperski, D. Lindenbaum, G. Pang, J. Huang, S. Basu, F. Hughes, D. Tuia, R. Raska, Deepglobe 2018: A challenge to parse the earth through satellite images, in: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE, 2018, pp. 172–181.
- [7] D. Dua, C. Graff, UCI Machine Learning Repository, 2017. <http://archive.ics.uci.edu/ml>.
- [8] G. Eibl, K.-P. Pfeiffer, Multiclass boosting for weak classifiers, *J. Mach. Learn. Res.* 6 (Feb) (2005) 189–210.
- [9] J.H. Friedman, Greedy function approximation: a gradient boosting machine, *Ann. Stat.* 29 (5) (2001) 1189–1232.
- [10] S. García, J. Luengo, F. Herrera, Tutorial on practical tips of the most influential data preprocessing algorithms in data mining, *Knowl. Based Syst.* 98 (2016) 1–29.
- [11] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, I.H. Witten, The WEKA data mining software: an update, *ACM SIGKDD Explor. Newslett.* 11 (1) (2009) 10–18.
- [12] H. He, W. Zhang, S. Zhang, A novel ensemble method for credit scoring: adaptation of different imbalance ratios, *Expert Syst. Appl.* 98 (2018) 105–117.
- [13] C. Huang, Y. Li, C. Change Loy, X. Tang, Learning deep representation for imbalance classification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 5375–5384.
- [14] M.F. Kabir, S. Ludwig, Classification of breast cancer risk factors using several resampling approaches, in: *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2018, pp. 1243–1248.
- [15] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, in: *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.
- [16] G. Lemaître, F. Nogueira, C.K. Aridas, Imbalanced-learn: a python toolbox to tackle the curse of imbalanced datasets in machine learning, *The J. Mach. Learn. Res.* 18 (1) (2017) 559–563.
- [17] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [18] X.-Y. Liu, J. Wu, Z.-H. Zhou, Exploratory undersampling for class-imbalance learning, *IEEE Trans. Syst. Man Cybern. Part B (Cybernetics)* 39 (2) (2008) 539–550.
- [19] N. Lunardon, G. Menardi, N. Torelli, Rose: a package for binary imbalanced learning, *R J.* 6 (1) (2014).
- [20] R. Luo, S. Dian, C. Wang, P. Cheng, Z. Tang, Y. Yu, S. Wang, Bagging of xgboost classifiers with random under-sampling and totem link for noisy label-imbalanced data, in: *IOP Conference Series: 3rd International Conference on Automation, Control and Robotics Engineering (CACRE 2018)*, volume 428, IOP Publishing, 2018, p. 012004.
- [21] B.W. Matthews, Comparison of the predicted and observed secondary structure of t4 phage lysozyme, *Biochim. Biophys. Acta (BBA)* 405 (2) (1975) 442–451.
- [22] W. McKinney, Pandas: a foundational python library for data analysis and statistics, *Python for High Performance and Scientific Computing*, 14, 2011.
- [23] A. Natekin, A. Knoll, Gradient boosting machines, a tutorial, *Front. Neurobot.* 7 (2013) 21.
- [24] D. Nielsen, Tree Boosting With XGBoost-Why Does XGBoost Win “Every” Machine Learning Competition?, Norwegian University of Science and Technology, 2016.
- [25] K.B.A. Omar, Xgboost and LGBM for Porto Seguros Kaggle Challenge: A Comparison, 2018. Semester Project, ETH Zurich.
- [26] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: machine learning in python, *J. Mach. Learn. Res.* 12 (Oct) (2011) 2825–2830.
- [27] H. Peng, F. Long, C. Ding, Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy, *IEEE Trans. Pattern Anal. Mach. Intell.* 27 (8) (2005) 1226–1238.
- [28] N. Ponomareva, S. Radpour, G. Hendry, S. Haykal, T. Colthurst, P. Mitrichev, A. Grushetsky, Tf boosted trees: a scalable tensorflow based framework for gradient boosting, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2017, pp. 423–427.

- [29] D.M. Powers, Evaluation: from precision, recall and f-measure to ROC, informedness, markedness and correlation, *J. Mach. Learn. Technol.* (2011).
- [30] L. Prokhorenkova, G. Gusev, A. Vorobev, A.V. Dorogush, A. Gulin, Catboost: unbiased boosting with categorical features, in: *Advances in Neural Information Processing Systems*, 2018, pp. 6638–6648.
- [31] C.O. Sakar, G. Serbes, A. Gunduz, H.C. Tunc, H. Nizam, B.E. Sakar, M. Tutuncu, T. Aydin, M.E. Isenkul, H. Apaydin, A comparative analysis of speech signal processing algorithms for parkinsons disease classification and the use of the tunable q-factor wavelet transform, *Appl. Soft Comput.* 74 (2019) 255–263.
- [32] Y. Sun, A.K.C. Wong, M.S. Kamel, Classification of imbalanced data: a review, *Int. J. Pattern Recognit Artif Intell.* 23 (4) (2009) 687–719.
- [33] S. Tyree, K.Q. Weinberger, K. Agrawal, J. Paykin, Parallel boosted regression trees for web search ranking, in: *Proceedings of the 20th international conference on World wide web*, ACM, 2011, pp. 387–396.
- [34] S. Van Der Walt, S.C. Colbert, G. Varoquaux, The numpy array: a structure for efficient numerical computation, *Comput. Sci. Eng.* 13 (2) (2011) 22.
- [35] C. Wang, S. Wang, F. Shi, Z. Wang, Robust propensity score computation method based on machine learning with label-corrupted data, *arXiv:1801.03132* (2018).
- [36] C.-W. Wang, Y.-C. Lee, E. Calista, F. Zhou, H. Zhu, R. Suzuki, D. Komura, S. Ishikawa, S.-P. Cheng, A benchmark for comparing precision medicine methods in thyroid cancer diagnosis using tissue microarrays, *Bioinformatics* 34 (10) (2017) 1767–1773.
- [37] X. Wang, P. Cheng, X. Liu, B. Uzochukwu, Focal loss dense detector for vehicle surveillance, in: *2018 International Conference on Intelligent Systems and Computer Vision (ISCV)*, IEEE, 2018, pp. 1–5.
- [38] J.T. Wassan, H. Wang, F. Browne, H. Zheng, A comprehensive study on predicting functional role of metagenomes using machine learning methods, *IEEE/ACM Trans. Comput. Biology Bioinform.* 16 (3) (2019) 751–763.
- [39] Y. Xia, C. Liu, N. Liu, Cost-sensitive boosted tree for loan evaluation in peer-to-peer lending, *Electron. Commer. Res. Appl.* 24 (2017) 30–49.
- [40] C. Zhang, J. Bi, S. Xu, E. Ramentol, G. Fan, B. Qiao, H. Fujita, Multi-imbalance: an open-source software for multi-class imbalance learning, *Knowl. Based Syst.* 174 (2019) 137–143.
- [41] T. Zhang, L.-Y. Hao, G. Guo, A feature enriching object detection framework with weak segmentation loss, *Neurocomputing* 335 (2019) 72–80.
- [42] Z. Zhao, H. Peng, C. Lan, Y. Zheng, L. Fang, J. Li, Imbalance learning for the prediction of n 6-methylation sites in mRNAs, *BMC Genom.* 19 (1) (2018) 574.